

1.) a. Constraint 1: Add artificial a_1
(=)

$$\rightarrow x_1 - x_2 + x_3 + a_1 = 2$$

Constraint 2: Add slack s_1
($\leq$)

$$\rightarrow x_1 + x_2 + 2x_3 + s_1 = 10$$

Constraint 3: Subtract surplus s_2 , add artificial a_2
($\geq$)

$$\rightarrow x_1 + x_2 - x_3 - s_2 + a_2 = 6$$

b. $\min w = a_1 + a_2 \quad == \quad \max w' = -(a_1 + a_2)$

Phase 1 standard form LP:

$$\begin{aligned} \text{Max } w' &= -a_1 - a_2 \\ \text{s.t.} \end{aligned}$$

$$x_1 - x_2 + x_3 + a_1 = 2$$

$$x_1 + x_2 + 2x_3 + s_1 = 10$$

$$x_1 + x_2 - x_3 - s_2 + a_2 = 6$$

$$x_1, x_2, x_3, s_1, s_2, a_1, a_2 \geq 0$$

c. Simplex Tableau:

	x_1	x_2	x_3	s_1	s_2	a_1	a_2	RHS
a_1	1	-1	1	0	0	1	0	2
s_1	1	1	2	1	0	0	0	10
a_2	1	1	-1	0	-1	0	1	6
w'	-2	0	0	0	1	-1	-1	-8

1C.

```
from scipy.optimize import linprog

c = [-2, 4, -7]
A_ub = [[1, 1, 2], [-1, -1, 1]] #  $x_1 + x_2 + 2x_3 \leq 10$ ,  $-x_1 - x_2 + x_3 \leq -6$ 
b_ub = [10, -6]

A_eq = [[1, -1, 1]] #  $x_1 - x_2 + x_3 = 2$ 
b_eq = [2]

#  $x_1 \geq 0$ ,  $x_2 \geq 0$ ,  $x_3 \geq 0$ 
x_bounds = [(0, None), (0, None), (0, None)]

result = linprog(c, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
                 bounds=x_bounds, method='simplex')

print(f"Optimal x1: {result.x[0]}")
print(f"Optimal x2: {result.x[1]}")
print(f"Optimal x3: {result.x[2]}")
print(f"Optimal value: {-result.fun}")

Optimal x1: 4.0
Optimal x2: 3.3333333333333333
Optimal x3: 1.3333333333333333
Optimal value: 4.0

C:\Users\sar-home\AppData\Local\Temp\ipykernel_35408\3956303430.py:13:
DeprecationWarning: `method='simplex'` is deprecated and will be
removed in SciPy 1.11.0. Please use one of the HiGHS solvers (e.g.
`method='highs'`) in new code.
    result = linprog(c, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
    bounds=x_bounds, method='simplex')
```

2B.

```
c_phase1 = [0, 0, 0, 0, 0, 1]
A_eq_phase1 = [[1, -1, -1, 0, 0, 1], [2, 3, 0, 1, 0, 0], [2, 1, 0, 0,
1, 0]]
b_eq_phase1 = [8, 24, 12]

x_bounds_phase1 = [(0, None), (0, None), (0, None), (0, None), (0,
None), (0, None)]

result_phase1 = linprog(c_phase1, A_eq=A_eq_phase1, b_eq=b_eq_phase1,
                        bounds=x_bounds_phase1, method="simplex")
```


Using x_1 as pivot ($w' = -2$)
min ratio

$$a_1: \frac{2}{1} = 2$$

$$a_2: \frac{6}{1} = 6$$

$$s_1: 10$$

$\Rightarrow a_1$ leaves basis

Pivot on $x_1 = 1$ in $R1$:

Tabikan $\rightarrow$	x_1	x_2	x_3	s_1	s_2	a_1	a_2	RHS
$R1:$	x_1	1	-1	1	0	0	1	2
$R2:$	s_1	1	1	2	1	0	0	10
$R3:$	a_2	1	1	-1	0	-1	0	6
$w':$	w'	-2	0	0	0	1	-1	-8

$$R2 \leftarrow R2 - R1$$

$$\rightarrow R2: [0, 2, 1, 1, 0, -1, 0 | 8]$$

$$R3 \leftarrow R3 - R1$$

$$\rightarrow R3: [0, 2, -2, 0, -1, -1, 1 | 4]$$

$$w' \leftarrow w' + 2R1$$

$$\rightarrow w': [0, -2, 2, 0, 1, 1, -1 | -4]$$

Using x_2 as pivot ($w' = -2$)
min ratio

$$s_1: \frac{8}{2} = 4$$

$$a_2: \frac{1}{2} = 2 \rightarrow a_2 \text{ leaves basis}$$

2.) a. Constraint 1: Subtract Surplus S_1 , add artificial a_1
($\geq$) $\rightarrow x_1 - x_2 - S_1 + a_1 = 8$

Constraint 2: Add Slack S_2
($\leq$) $\rightarrow 2x_1 + 3x_2 + S_2 = 24$

Constraint 3: Add Slack S_3
($\leq$) $\rightarrow 2x_1 + x_2 + S_3 = 12$

Standard Form: Max $z = 2x_1 + x_2$
 $s.t.$

$$x_1 - x_2 - S_1 + a_1 = 8$$

$$2x_1 + 3x_2 + S_2 = 24$$

$$2x_1 + x_2 + S_3 = 12$$

$$x_1, x_2, S_1, S_2, S_3, a_1 \geq 0$$


```

print(f"Optimal x1: {result_phase1.x[0]}")
print(f"Optimal x2: {result_phase1.x[1]}")
print(f"Optimal x3: {result_phase1.x[2]}")
print(f"Optimal value: {result_phase1.fun}")

```

```

Optimal x1: 6.0
Optimal x2: 0.0
Optimal x3: 0.0
Optimal value: 2.0

```

C:\Users\sar-home\AppData\Local\Temp\ipykernel_35408\4083208617.py:7:
DeprecationWarning: `method='simplex'` is deprecated and will be removed in SciPy 1.11.0. Please use one of the HiGHS solvers (e.g. `method='highs'`) in new code.

```

result_phase1 = linprog(c_phase1, A_eq=A_eq_phase1,
b_eq=b_eq_phase1, bounds=x_bounds_phase1, method="simplex")

```

2C.

```

c_phase2 = [-2, -1, 0, 0, 0]
# x1 - x2 - s1 = 8, 2x1 + 3x2 + s2 = 24, 2x1 + x2 + s3 = 12
A_eq_phase2 = [[1, -1, -1, 0, 0], [2, 3, 0, 1, 0], [2, 1, 0, 0, 1]]
b_eq_phase2 = [8, 24, 12]

```

```

x_bounds_phase2 = [(0, None), (0, None), (0, None), (0, None), (0,
None)]

```

```

result_phase2 = linprog(c_phase2, A_eq=A_eq_phase2, b_eq=b_eq_phase2,
bounds=x_bounds_phase2, method="simplex")

```

```

if not result_phase2.success:
    print("Problem is infeasible!!!")
else:
    print(f"Optimal x1: {result_phase2.x[0]}")
    print(f"Optimal x2: {result_phase2.x[1]}")
    print(f"Optimal x3: {result_phase2.x[2]}")
    print(f"Optimal value: {result_phase2.fun}")

```

Problem is infeasible!!!

C:\Users\sar-home\AppData\Local\Temp\ipykernel_35408\2265422950.py:8:
DeprecationWarning: `method='simplex'` is deprecated and will be removed in SciPy 1.11.0. Please use one of the HiGHS solvers (e.g. `method='highs'`) in new code.

```

result_phase2 = linprog(c_phase2, A_eq=A_eq_phase2,
b_eq=b_eq_phase2, bounds=x_bounds_phase2, method="simplex")

```